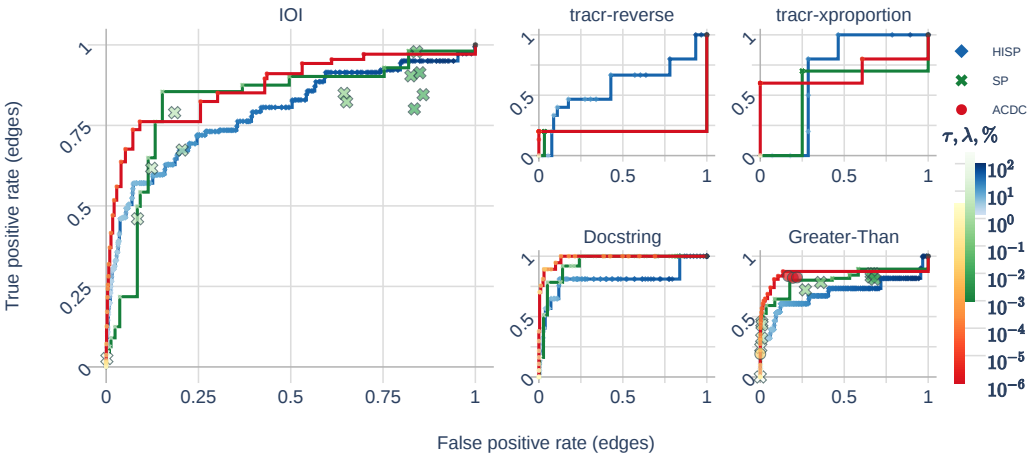

Figure 3: ROC curves of ACDC, SP and HISP identifying model components from previous work, across 5 circuits in transformers. The points on the plot are cases where SP and ACDC return subgraphs that are not on the Pareto frontier. The corresponding AUCs are in Table 2.

We sweep over a range of ACDC thresholds τ, SP regularization parameters λ, or number of HISP elements pruned k. We plot pessimistic segments between points on the Pareto frontier of TPR and FPR, over this range of thresholds (Fawcett, 2006). ACDC and SP optimize the KL divergence for tasks where this makes sense (all but `tracr` tasks, which use the L2 distance). All methods employ activations with corrupted data. Appendix C describes and Appendix E experiments with different design choices for the metric and activation patching methodology.

Figure 3 shows the results of studying how well existing methods recover circuits in transformers. We find that i) methods are very sensitive to the corrupted distribution, ii) ACDC has competitive performance (as measured by AUC) with gradient-descent based methods iii) ACDC is not robust, and it fails at some settings.

Several of the tasks appeared to require specific distributions and metrics for the areas under the curves to be large. For example, ACDC achieved poor performance on both `tracr` tasks in Fig. 3, but the circuit was perfectly recovered by ACDC at any threshold $\tau > 0$ when patching activations with zeros (Appendix I). Furthermore, ACDC achieves a greater AUC on the IOI and Greater-Than and tracr-reverse tasks than both of the other methods, and hence overall is the optimal algorithm. As an example of the variable performance of circuit recovery algorithms, on the Docstring task we achieve the high perfomance when using the ACDC algorithm with the docstring metric (Appendix H). However in other tasks such as the IOI task, ACDC performance was worse when optimizing for logit difference.

Further research in automated interpretability will likely yield further improvements to the FPR and TPR of circuit discovery. We outline limitations with all current methods, but also gesture at likely fundamental limitations of the false positive and true positive measures. A limitation with all existing methods is that they optimize a single metric. This means they systematically miss internal model components such as the "negative" components found in previous work (IOI, Docstring) that are actively harmful for performance. The IOI recovery runs were not able to recover negative heads when optimizing for logit difference. Even when optimizing for low KL divergence, the negative components were only recovered when very small thresholds were used (Figure 15).

Additionally, a more fundamental limitation to measuring the false and true positive rates of circuit recovery methods is that the ground-truth circuits are reported by practitioners and are likely to have included extraneous edges and miss more important edges. The language model circuits studied in our work (Appendices F-H) involve a large number of edges (1041 in the case of IOI) and the full models contain more than an order of magnitude more edges. Since these interpretability works are carried out by humans who often report limitations of their understanding, our 'ground-truth' is not 100% reliable, limiting the strength of the conclusions that can be drawn from the experiments in this section.

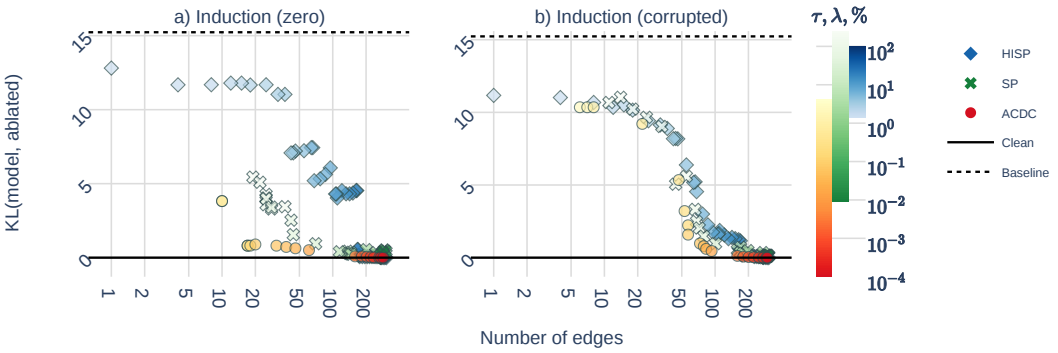

Figure 4: Comparison of ACDC and SP with both zero-input activations (left) and corrupted activations (right). We plot the KL Divergence on a held-out test set against the number of edges of each hypothesized circuit. Lower KL divergence and fewer edges correspond to better subgraphs. Darker points include more edges in the hypothesis: they use smaller ACDC τ, smaller SP regularization λ or a higher percentage of nodes in HISP.

4.2 Stand-alone circuit properties with a test metric

This section evaluates the algorithms by studying the induction task. We measure the KL Divergence of the circuits recovered with the three methods to the original model. This is an indirect measure of **Q1**, with the advantage of not relying on the completeness or correctness of previous works. As an indicator of **Q2**, we also measure the number of edges that a hypothesized circuit contains. A circuit with fewer edges which still obtains a low KL Divergence is less likely to contain components that do not participate in the behavior. In Appendix L we also introduce and explain experiments on **reset networks** that provide more evidence for **Q2**.

Our mainline experimental setup is to run the circuit recovery algorithms as described in Algorithm 1 and Section 3 and then measure the KL Divergence for these circuits on the induction task (Appendix J). In brief, ACDC performs better that the other methods under these experimental conditions with both corrupted and zero activations. For example, the left-hand side of Figure 4 shows that, above 20 edges, ACDC starts having a slight advantage over other methods in terms of behavior recovered per number of edges as all points on the Pareto-frontier with at least this many edges are generated from ACDC runs. Appendix E describes many further experiments with variations on setup to provide a more complete picture of the performance of the circuit recovery algorithms. For example, when we measure the loss (the task-specific induction metric; Table 1) of subgraphs recovered by optimizing KL Divergence, we find very similar qualitative graphs to Figure 4.

In Appendix L we see that the KL divergence that all methods achieve is significantly lower for the trained networks, indicating that all the methods get signal from the neural network's ability to perform induction (Figure 4). HISP and SP with zero activations, and to some extent SP with corrupted activations are also able to optimize the reset network. This suggests that these methods are somewhat more prone to finding circuits that don't exist (i.e. evidence against **Q2**).

5 Related work

Mechanistic interpretability encompasses understanding features learnt by machine learning models (Olah, Mordvintsev, and Schubert, 2017; Elhage et al., 2022), mathematical frameworks for understanding machine learning architetures (Elhage et al., 2021) and efforts to find *circuits* in models (Nanda et al., 2023; Cammarata et al., 2021; Chughtai, Chan, and Nanda, 2023; Wang et al., 2023). The higher standard of a mechanistic understanding of a model has already had applications to designing better architectures (Fu et al., 2023), though the speculative goal of mechanistic interpretability is to understand the behavior of whole models, perhaps through describing all their circuits and how they compose. Little work has been done to automate interpretability besides Bills et al. (2023) who use language models to label neurons in language models.

Neural network pruning masks the weights of neural networks to make their connectivity more sparse (LeCun, Denker, and Solla, 1989). In contrast to our aims, the pruning literature is typically concerned with compressing neural networks for faster inference or to reduce storage requirements (Wang, Wohlwend, and Lei, 2020; Kurtic et al., 2022). Early work (Hassibi and Stork, 1992) hoped pruning would lead to more interpretable networks, but progress towards interpretability via pruning is limited (Grover, Gawri, and Manku, 2022).

Pruning techniques may learn masks from data, which is a special case of more generally using gradient information. Masks can also be learned from data, with an objective function that balances model performance and network sparsity (Louizos, Welling, and Kingma, 2018; Wang, Wohlwend, and Lei, 2020; Cao, Sanh, and Rush, 2021). This is a useful comparison to ACDC as learnable masks do not change the weights of our model after pruning (Frantar and Alistarh, 2023). Examples of gradient information being used more generally includes Michel, Levy, and Neubig (2019) who decide which heads should be pruned by using the absolute value of their gradients, while "movement pruning" (Sanh, Wolf, and Rush, 2020) removes parameters that have high velocity to a low magnitude. ACDC is different from pruning and other compression techniques (Zhu et al., 2023) since i) the compressed networks we find are reflective of the circuits that model's use to compute outputs to certain tasks (Section 4) and ii) our goal is not to speed up forward passes, and generally our techniques slow forwards passes.

Causal interpretation. Much prior research on understanding language models has drawn inspiration from causal inference (Pearl, 2009), leading to the development of frameworks that provide causal explanations for model outputs (Pearl, 2009; Feder et al., 2021; Geiger et al., 2021; Wu et al., 2022; Kaddour et al., 2022). Other work (Vig et al., 2020) discusses the difference between indirect effects and direct effects inside language models, and experiments on removing subsets of these heads using heads' direct effects as proxies for the overall contribution of these heads. Goldowsky-Dill et al. (2023) introduce 'path patching' to analyze the effects of different subsets of edges in computational graphs of models. In parallel to our work, Wu et al. (2023) develop a method to automatically test whether neural networks implement certain algorithms with causal testing. Our work is focused on finding rather than verifying an outline of an algorithm implemented by a model.

Computational subgraphs for interpretability. Training dynamics in residual models can be explained by shallow paths through the computational graph (Veit, Wilber, and Belongie, 2016). MLP layers can be modelled as memory that is able to represent certain properties of the network inputs (Geva et al., 2021). Residual transformer models have been modelled as the sum of all different paths through the network (Elhage et al., 2021). Later work has used insights from looking at subgraphs of models in order to edit models' behaviors (Bau et al., 2020; Meng et al., 2022) and test interpretability hypotheses (Chan et al., 2022).

6 Conclusion

We have identified a common workflow for mechanistic interpretability. First, pin down a behavior using a metric and data set. Second, conduct activation patching experiments to understand which abstract units (e.g. transformer heads) are involved in the behavior. Third, iterate the previous steps with variations of the behavior under study, until the model's algorithm is understood.

The main proposed algorithm, ACDC, systematically conducts all the activation patching experiments necessary to find which circuit composed of abstract units is responsible for the behavior. We have shown that ACDC and SP recover most of the compositional circuit that implements a language model behavior, as judged by comparison to previous mechanistic interpretability work (Section 4). ACDC with zero activations fully recovers the circuit of toy models (Fig. 9). Further, there is early evidence of the use of ACDC to help with novel interpretability work, discovering a surprising outline of a subgraph of GPT-2 Small that predicts gendered pronoun completion (Appendix K). Here, practitioners used ACDC to generate a subgraph including the most important pathway through a model's computation, and checked that this reflects the model's computation in normal (unablated) forward passes. This surprising find was an early example of the summarization motif (Tigges et al., 2023).

However, both ACDC and SP have limitations which prevent them from fully automating step 3 of the identified workflow (activation patching). First, they tend to miss some classes of abstract units that are part of the circuit, for example the negative name mover heads from IOI (Wang et al., 2023).